

Structure-Faithful PDF Ingestion for Tree-Based RAG

Federico Mizzaro, Omar Chafik, Abdul Rahim

June 26, 2026

Motivation

Hospital infection-prevention and control (IPC) teams rely on multi-level clinical guidelines to make (time-sensitive) decisions. Recent vectorless RAG frameworks such as PageIndex[1] would be able to make use of such document hierarchy for reasoning-based, top-down retrieval boasting extreme accuracy[2]. However, these tools assume clean, correctly-structured input of the document set: the heading hierarchy and leaf content must survive ingestion intact. The PageIndex documentation itself warns that for markdown converted from PDF, the de facto format for published and archived documents, “most existing conversion tools cannot preserve the original hierarchy” [3].

The PageIndex ecosystem’s own answer to this problem, used by the OpenKB knowledge base[4], converts documents with Microsoft’s `markdown` and extracts images with `pymupdf`. For “large or complex PDFs,” both projects direct users to the proprietary, hosted *PageIndex Cloud* OCR service. No open-source, locally-deployable OCR ingestion path exists yet in this ecosystem.

For clinical deployment, a cloud OCR dependency is questionable at best, and the quality gap left by text-layer extraction alone is a patient-safety risk. Our target deployment is therefore deliberately constrained: **commodity, CPU-only, fully local** — no GPU, no hosted service. This project addresses the gap directly: we build an open ingestion path on top of the PaddleOCR-VL-1.6 [5] model family, and we supply the reproducible fidelity benchmark that the ecosystem currently lacks to measure the quality of such a path.

Analysis

The project has two separable contributions.

Contribution A: a benchmark harness. We construct a ground-truth corpus of structured PDFs by compiling LaTeX documents to PDF while retaining the source. Ground-truth heading trees are extracted directly from `\section`, `\subsection` (and so on) commands, and ground-truth asset inventories from (as examples) `\includegraphics` and `\begin{table}` declarations. This gives us unambiguous, machine-readable ground truth without human annotation. The harness then runs competing ingestion paths — the OpenKB-style `markdown`+LLM reconstruction (the current default) and our OCR pipeline — and compares the resulting trees against ground truth. A central methodological commitment is that we score *detection* and *nesting* **separately** (see Implementation), so a win in finding the right headings is never confused with a win in nesting them correctly.

Contribution B: an open, CPU-tractable OCR ingestion pipeline. The naive use of PaddleOCR-VL — running the 0.9B vision-language model over every block — is far too slow to be local and practical (a single 14-page paper took ~ 48 min on our CPU-only target). Our pipeline instead recovers the heading *tree* with a fast detection model and a lightweight recogniser, reserving the heavy VLM for optional content recognition. On top of detection we add a *leveling* layer that turns flat heading blocks into a relative hierarchy, driven primarily by the headings’ own

rendered numbering and falling back to a font-size signal when numbering is absent. A planned third stage (Contribution B', below) introduces an *anchored-asset leaf* scheme that attaches figures and tables as dedicated, addressable leaf nodes in the PageIndex tree rather than inline blobs the LLM happens to read.

Implementation

Main libraries: `paddlex/paddleocr` (PP-DocLayoutV3 + mobile PP-OCR), `pypdfium2`, `markitdown`, and the open-source PageIndex codebase.

Document layout analysis (the speed decision). We found empirically that the VLM's cost is dominated by *autoregressive text generation* over every block, not vision encoding — shrinking the input image made it *slower*, and no single knob (`max_pixels`, `max_new_tokens`, disabling sub-pipelines) made it tractable. The win is architectural: for the heading tree, only the short *title* blocks need to be read. We therefore use PP-DocLayoutV3, a detection model with *no* generation, to label every block (`doc_title`, `paragraph_title`, `text`, `figure_title`, `table`, ...) with a bounding box at $\sim 1\text{--}4\text{s}/\text{page}$, and a mobile PaddleOCR (classic lite det+rec) to recognise only the title crops — at *identical* title quality to the VLM. This is one to two orders of magnitude faster than the full VLM, which remains available for the separate, slower content-fidelity pass.

Heading detection, not heading levels. The layout model gives a reliable *label* but not a reliable level. We isolate real section headings by label alone (`paragraph_title`), with *no* text heuristics: captions carry a distinct `figure_title` label, so filtering on the label — rather than a rule like “drop titles starting with *Table*” — avoids overfitting the evaluator to any one document. (A rare caption mislabelled `paragraph_title` is then an honest false positive, not a hidden patch.)

Leveling. Levels are reconstructed from the signal already present in the recognised title text: its **rendered numbering**. Dotted arabic (1, 1.1, 1.2.3) gives level by component count; IEEE-style I./A./1) maps to section/subsection/subsubsection. The hard case — single tokens like I, V, C, D that are valid roman numerals *and* subsection letters, made worse because IEEEtran renders sections in Title Case (so capitalisation is uninformative) — is resolved by **sequence**: a token is a section if it continues the roman run, a subsection if it continues the letter run. For *unnumbered* documents we fall back to an independent **text-layer font-size** signal read via `pypdfium2` (largest size = level 1); font size is unreliable in general (it inverts on IEEE small-caps) but is safe here precisely because numbered docs take the numbering path, so the font fallback only fires where size decreases cleanly with depth. Throughout, we interpret the heading sequence as a *relative* hierarchy: a level gap (a heading nested more than one step below its parent, possibly an authoring artefact) is absorbed as the next nesting step, not treated as an error to repair. A column-aware **reading-order** sort is applied first, since two-column layouts otherwise return blocks out of order (e.g. placing 2.1 before section 2).

Baseline. The comparison point is the OpenKB/PageIndex path: `markitdown` dumps the PDF text layer to flat markdown (no headings) and an LLM reconstructs the heading list. We make this a *strong* and *deterministic* baseline — the reconstruction is produced by a capable LLM **blind to the LaTeX ground truth**, cached as one frozen artefact per document, so runs are reproducible and we are beating the conservative target rather than a weak local model.

Benchmark computation. LaTeX source is parsed to ground-truth trees and asset inventories. We replace a single tree-edit-distance score with two decomposable F1 metrics over normalised titles: `title_f1`, the level-free bag overlap of section titles (pure *detection*), and `edge_f1`, the overlap of parent→child pairs (*nesting* and identity together). Both use a `normalise_title` step that strips rendered numbering and markdown emphasis, and both are computed identically for every condition. Content fidelity (recognising body prose under each heading) is a separate, slower concern handled by the full VLM pass and scored by token-level edit distance over aligned

leaves; the fast structure path deliberately does not attempt it. OCR results are cached by (`pdf`, `config`); we verified the OCR output is bit-stable across repeated runs, so the benchmark is fully reproducible. The ingest module slots into the existing PageIndex `modules/ingest/` registry, altering only how the tree is built and leaving retrieval untouched: the goal is not to improve retrieval but to make the OCR path open, local, and measurable.

Contribution B' (planned): anchored assets. Figures and tables will be attached as dedicated leaf nodes anchored to the prose leaf under which they appear (by spatial association from the layout bounding boxes), making non-text content a first-class, addressable part of the retrieval tree. This stage is designed but not yet implemented; the evaluation below treats it as the final ablation step.

Evaluation

We evaluate ingestion fidelity comparatively, against the OpenKB `markdown`+LLM reconstruction baseline, on a LaTeX ground-truth corpus spanning the layout and numbering axes: synthetic documents in distinct templates (single- and two-column `article` with arabic numbering, `IEEEtran` with roman/letter/paren numbering, and unnumbered `article` both flat and three-level nested), plus real two-column IEEE papers — seven documents in total, each compiled from source with `tectonic`. The design is a four-condition ablation that isolates each component: (A) `markdown`+LLM (OpenKB default); (B) OCR title detection with *no* leveling (flat tree); (C) OCR with numbering/font leveling; (D) the full pipeline including anchored-asset leaves. Detection is scored by `title_f1` (A vs. B), nesting by `edge_f1` (C), and the asset contribution by the (C)→(D) contrast, measured as asset recall: the fraction of `\includegraphics` / `\begin{table}` declarations recovered as anchored leaves.

This design is suitable because it is fully automatable, needs no subjective human annotation, compares *text against text* at every stage (avoiding the ambiguity of PDF-versus-text comparison), and directly measures the gap between the current open-source default and our pipeline — attributing each gain to a specific component rather than to the pipeline as a whole.

Early results on the assembled corpus already support the core thesis. On a hard real two-column IEEE paper, OCR recovers *every* heading (`title_f1` $\approx$ 0.97, recall 1.0) where the `markdown` baseline reaches only $\approx$ 0.64 (recall 0.53): text-layer extraction interleaves the two columns and even drops inter-word spaces, scattering the 1)/2) subsubsections beyond recovery, whereas reading the rendered image per block isolates each heading regardless of column flow. On a clean single-column document the two tie at 1.0 — OCR earns its cost on hard layouts, not on easy ones, which we state plainly. With leveling enabled, five of the seven documents reconstruct the tree *exactly* (`edge_f1` = 1.0). The two residual cases are **characterised, not patched**: `IEEEtran` renders `\subsubsection` run-in (inline, so those headings do not exist as blocks in the rendered PDF — a rendering boundary OCR cannot cross), and one table caption mislabelled by the layout model cascades into its neighbours' nesting. We deliberately do not add document-specific rules to erase these, so the reported numbers reflect the pipeline's true behaviour rather than a tuned evaluator.

References

- [1] Vectify AI, *PageIndex: Reasoning-based RAG with hierarchical document trees*, <https://github.com/VectifyAI/PageIndex>, TODO: replace with canonical BibTeX from the PageIndex repo, 2025.
- [2] Vectify AI, *PageIndex retrieval benchmark*, <https://github.com/VectifyAI/PageIndex>, TODO: cite the specific reported benchmark (e.g. FinanceBench), 2025.
- [3] Vectify AI, *PageIndex*, <https://github.com/VectifyAI/PageIndex>, 2025.

- [4] Vectify AI, *OpenKB: An open knowledge base for PageIndex*, <https://github.com/VectifyAI/OpenKB>, 2025.
- [5] PaddlePaddle Team, “PaddleOCR-VL: A vision-language model for document parsing,” 2026, Model: <https://huggingface.co/PaddlePaddle/PaddleOCR-VL-1.6>. arXiv: 2606.03264 [cs.CV].